Chapter 15

Fraud Detection, Compliance, and Financial Crime

15.1 Introduction

Chapter 1 previewed this chapter with a single number: a fraud detection model that correctly flags 95% of actual fraud and mistakenly flags only 3% of legitimate transactions still produces a flagged transaction that is fraudulent only about 24% of the time, because fraud is rare enough that even a small false-positive rate generates many more false alarms than true detections. That number is the central operating constraint of every fraud detection and anti-money-laundering (AML) system, and this chapter is built around managing it. Chapter 6 developed classification models and the confusion matrix using loan default as the running example; Chapter 9 applied logistic regression, scoring, and rating models to credit risk. This chapter applies the same machinery to fraud, laundered money, and other financial crime. There the base rate is typically far lower than a loan portfolio's default rate, and the consequences of a false positive or a missed detection differ.

Four worked examples run through the chapter. Chapter 1's Bayes' theorem calculation is scaled up to a day's transactions, turning a posterior probability into an alert queue a compliance team must work through. A small fraud-scoring logistic regression shows how a few transaction features combine into a fraud probability, and where such a model still misses a case. A Kaplan-Meier survival analysis, new to this book, estimates how long flagged accounts stay active before fraud is confirmed, introducing censored observations. Finally, a structuring example shows how AML rules are built around a legal reporting threshold, and how criminals evade it.

15.2 Detecting Fraud

Fraud is a rare-event classification problem, and this part builds a detector while confronting how rare the positives are.

15.2.1 The Landscape of Financial Crime and Where AI Fits

Suppose a bank builds a fraud-scoring model that catches nearly every stolen-card transaction and considers its detection obligations satisfied, only to face a regulatory fine for missing a money-laundering scheme the model was never designed to catch. "Fraud detection" understates the breadth of what financial institutions are legally required to monitor for.

Table 15.1 organizes the main categories of financial crime this book touches on, the detection approach each typically relies on, and where it is developed.

Every row in Table 15.1 shares the same statistical structure: a rare, adversarial event must be distinguished from an overwhelming volume of legitimate activity, and the adversary adapts once a detection method becomes known. A credit-scoring model's borrowers, Chapter 9's subject, are not trying to defeat the model; a money launderer designing a transaction pattern to stay under a reporting threshold is. Financial crime

Wulin.Suo@Queensu.ca

Crime type	Typical detection approach	Developed in
Card and payment fraud	Real-time transaction scoring, anomaly detection	Sections 15.2.2, 15.2.3
Account takeover	Behavioral biometrics, device and login anomaly detection	Section 15.4.1
Synthetic identity fraud	Cultivation-pattern and shared-attribute linkage; unsupervised detection where labels are absent	Section 15.3
Authorized push payment (scam) fraud	Deviation from the customer's own payment history; mule detection on the receiving side	Section 15.3
Money laundering	Rule-based transaction monitoring, network analysis, structuring detection	Section 15.5.1
Market manipulation (spoofing, layering)	Order-book pattern surveillance	Chapter 14's spoofing case vignette
Insider trading	Anomaly detection linking trading activity to material news events	Chapter 1's Bayesian updating, extended here

Table 15.1: A taxonomy of financial crime and detection approaches

detection therefore combines the statistical tools of Chapters 6 and 9 with rules, network analysis, and a regulatory reporting apparatus with no counterpart in credit scoring.

A second difference between the rows decides more about which methods are available than the crime type does: what *labels* each problem produces as a by-product of its operation. Card fraud sits at one extreme. A cardholder disputes a charge within days, the dispute is ground truth rather than an opinion, and millions arrive automatically, so a supervised model can be retrained continuously against real outcomes. Money laundering sits at the other. The only label a bank ordinarily obtains is that one of its own analysts filed a Suspicious Activity Report, which records a suspicion rather than a confirmed crime, and law enforcement almost never reports back what came of it. A model fitted to those filings learns its institution's alerting behavior rather than criminality, and the true prevalence of laundering, the base rate every calculation in the next section depends on, is unmeasurable.

This asymmetry is why the methods in this chapter are distributed as they are. Supervised learning dominates where outcomes are observed; unsupervised detection, rules, and network structure carry the weight where they are not; and machine learning enters anti-money laundering largely through alert triage, ranking work the rules have already produced, because analyst dispositions are the one label that problem generates in quantity. Ask of each method below what it is trained on, and whether that is an outcome or an opinion.

15.2.2 Anomaly Detection in Transactions

Suppose a bank wants to stop a stolen credit card from being drained before the cardholder notices it is missing, deciding in real time whether each new charge looks like the cardholder or a thief. The simplest systems built for this decision are rule-based: a transaction is flagged if it exceeds a fixed dollar threshold, occurs in a country the cardholder has never transacted in, or arrives within seconds of another transaction from the same account in a distant location (a velocity check).

Rule-based systems are transparent and easy to audit, which matters for the regulatory reasons discussed later, but they are brittle: a threshold set to catch fraud will also catch a legitimate customer's unusual but innocent purchase. A publicly known threshold (such as the $10,000 reporting trigger of Section 15.5.1) can be engineered around.

Wulin.Suo@Queensu.ca

Statistical anomaly detection asks instead whether a transaction is unusual relative to the customer's own history, computing something like a z-score for amount, location, or timing against that customer's rolling mean and standard deviation, or using Chapter 6's unsupervised clustering to flag transactions that resemble none of the customer's usual behavioral clusters. Both approaches produce the same output as Chapter 6's classifiers, a binary flag or a risk score thresholded into one, and so face the same trade-off Chapter 1's Bayes' theorem example quantified.

Worked Example 15.1: The Bayes' Theorem Result at Full Transaction Volume

Chapter 1's example assumed a bank where 1% of transactions are fraudulent, a model correctly flags 95% of actual fraud, and incorrectly flags 3% of legitimate transactions, and found that a flagged transaction is fraudulent only about 24.2% of the time. That posterior's practical significance is easy to underestimate until it is scaled up to the volume a real institution processes.

Suppose the bank handles 500,000 transactions in a day. At a 1% fraud rate, this implies 5,000 fraudulent and 495,000 legitimate transactions, and applying the model's 95% true-positive rate and 3% false-positive rate gives

$$TP = 5{,}000 \times 0.95 = 4{,}750, \qquad FN = 5{,}000 \times 0.05 = 250,$$
$$FP = 495{,}000 \times 0.03 = 14{,}850, \qquad TN = 495{,}000 \times 0.97 = 480{,}150.$$

The model flags $4{,}750 + 14{,}850 = 19{,}600$ transactions a day, of which only 4,750 are fraud, giving

$$\text{Precision} = \frac{4{,}750}{19{,}600} \approx 0.2423, \qquad \text{Recall} = \frac{4{,}750}{5{,}000} = 0.95,$$

matching, up to rounding, Chapter 1's 24.2% posterior probability, since precision and $\Pr(\text{Fraud} \mid \text{Flag})$ are the same quantity computed two ways. The F_1 score, $2 \times 0.2423 \times 0.95/(0.2423 + 0.95) \approx 0.386$, is a useful single-number summary for comparing models, but it is the raw count, 19,600 flagged transactions in one day, that a compliance team must staff and review; Section 15.6.1 returns to this cost.

This is why fraud detection systems almost never rely on a single model or threshold. A bank that lowered its false-positive rate to improve precision would, by the same arithmetic, also lower recall and let more fraud through, unless it also improved the model's discriminative power. That is the motivation for combining distinct signals (transaction-level scoring, behavioral risk tiering, and survival modeling) rather than relying on any one alone.

A Second Application: Insider Trading Surveillance

The same Bayesian structure applies beyond transaction fraud. Market surveillance teams monitor for insider trading (trading ahead of material non-public information) largely by flagging abnormal trading volume or price movement in the days before a public announcement (an earnings release, a merger, a regulatory decision). This is the anomaly-detection logic of Chapter 14's order-flow-imbalance model, applied to a much lower-frequency, event-driven setting.

Suppose that, across all announcements a surveillance team reviews in a year, insider trading precedes about 2% of them, a volume-anomaly detector flags 75% of these cases, and it also flags 8% of announcements with no insider trading, a higher false-positive rate than Section 15.2.2's transaction-fraud example because pre-announcement volume is noisier than a single card transaction. Applying Bayes' theorem as in Chapter 1,

$$\Pr(\text{Flag}) = 0.75(0.02) + 0.08(0.98) = 0.0934, \qquad \Pr(\text{Insider} \mid \text{Flag}) = \frac{0.75(0.02)}{0.0934} \approx 0.1606,$$

so a flagged announcement reflects insider trading only about 16.1% of the time, even lower than Section 15.2.2's 24.2%. The interesting question is *why*: this example's true-positive rate (75%) is worse than the transaction-fraud example's (95%), which alone would push precision down only modestly, but its false-positive rate (8%) is more than double the other's (3%), and that difference dominates.

At a low base rate, precision is far more sensitive to the false-positive rate than to the true-positive rate, since the false-positive rate is multiplied by the (much larger) population of non-events; design effort is therefore almost always better spent tightening false-positive rates than chasing marginal gains in true-positive rates, a lesson that applies equally to the transaction-fraud and AML alert systems in this chapter.

Wulin.Suo@Queensu.ca

15.2.3 Worked Example 15.2: Fraud Scoring with Logistic Regression

Section 15.2.2's example treated the detection model as a black box with a fixed true- and false-positive rate; this section builds one explicitly, using the logistic regression Chapter 6 introduced and Chapter 9 applied to credit scoring. Table 15.2 records ten transactions, each with three features: the transaction amount expressed as a z-score relative to the cardholder's typical spending, the distance from the cardholder's home address in hundreds of kilometers, and a late-night indicator (1 if the transaction occurred between midnight and 5 a.m. local time), together with whether the transaction was fraudulent.

Transaction	Amount z-score	Distance (100s km)	Late night	Actual fraud	Predicted
1	3.1	8.0	1	1	1
2	0.2	0.1	0	0	0
3	2.8	6.5	1	1	1
4	1.9	6.0	0	0	0
5	0.4	0.2	0	0	0
6	3.4	9.2	1	1	1
7	0.1	0.4	0	0	0
8	0.6	0.1	1	0	0
9	1.2	0.3	1	1	0
10	0.3	0.2	0	0	0

Table 15.2: Ten transactions used to fit a fraud-scoring logistic regression

Fitting a logistic regression

$$\Pr(\text{Fraud}) = \sigma(\beta_0 + \beta_1 z + \beta_2 d + \beta_3 n) \tag{15.1}$$

using the sigmoid function introduced in Chapter 6, to these ten observations gives coefficients $\hat{\beta}_0 \approx -2.57$, $\hat{\beta}_1 \approx 0.83$ (amount z-score), $\hat{\beta}_2 \approx 0.15$ (distance), and $\hat{\beta}_3 \approx 0.82$ (late night), all with the intuitive sign: an unusually large purchase, far from home, late at night, each raises the predicted fraud probability.

These coefficients carry one caveat. The ten transactions are linearly separable (a linear support vector machine classifies all ten correctly), and on separable data plain maximum likelihood has no finite solution, because scaling up any separating coefficient vector always raises the likelihood. The fit above therefore uses scikit-learn's default L2 (ridge) penalty with $C = 1$, which shrinks the coefficients toward zero as Chapter 6's ridge regression shrinks its slopes, acting like a prior that keeps a ten-point model from claiming certainty it has not earned. Weakening the penalty shows what it is holding back: at $C = 10^6$ the coefficients run to $(27.08, -4.15, 12.70)$ with an intercept of -36.10, every transaction is classified correctly, and the distance coefficient even turns negative. The penalized fit is the more sensible model, but transaction 9's miss below is partly a product of the penalty, not of the data alone.

At a 0.5 threshold, the model flags transactions 1, 3, and 6 (true positives) and clears 2, 4, 5, 7, 8, and 10 (true negatives), but assigns transaction 9 a predicted probability of only about 33%, missing a fraud case (a false negative) despite the late-night flag, because its amount and distance were unremarkable. There are no false positives in this small sample, giving

$$\text{Precision} = \frac{3}{3+0} = 1.00, \qquad \text{Recall} = \frac{3}{3+1} = 0.75,$$

$$F_1 = \frac{2(1.00)(0.75)}{1.00 + 0.75} \approx 0.857,$$

a recall figure that, by design, mirrors Chapter 6's loan-default confusion matrix: three true positives and one false negative out of four positive cases.

Transaction 9 illustrates a structural limitation: a fraud that looks unremarkable in isolation will not be caught by a model that looks at one transaction at a time, however well it is fit. That is why Section 15.4.1 layers an account-level behavioral risk score on top of transaction scoring, and why Section 15.4.2 tracks how account risk accumulates over time.

Wulin.Suo@Queensu.ca

15.2.4 The Threshold Choice: A Precision-Recall Trade-off

The 0.5 threshold is a modeling choice, not a mathematical requirement, and moving it trades precision against recall as Chapter 6 described. Table 15.3 recomputes the same ten-transaction confusion matrix at two alternative thresholds.

Threshold	Precision	Recall	F_1
0.30	0.800	1.000	0.889
0.50	1.000	0.750	0.857
0.85	1.000	0.500	0.667

Table 15.3: Precision-recall trade-off at three classification thresholds

Lowering the threshold to 0.30 also flags transaction 4 (predicted probability 0.470, legitimate), a false positive, but catches transaction 9 as well, raising recall to 100% while precision falls from 1.00 to 0.80. Raising the threshold to 0.85 does the opposite: transaction 3 (predicted probability 0.822) falls below the bar and becomes a second false negative, so recall drops to 50% while precision stays perfect. Neither threshold is objectively correct: a bank sets its threshold by weighing the cost of a missed fraud against the cost of another false-positive investigation, not by any property of the model.

The choice becomes determinate once a cost is attached to each kind of error. Suppose reviewing a flagged transaction costs \$50 of investigator time and a missed fraud costs \$200 on average, so that operating at threshold τ costs

$$C(\tau) = \$50 \times (\text{transactions flagged}) + \$200 \times (\text{frauds missed}). \tag{15.2}$$

On the ten fitted probabilities this gives \$350 at the default $\tau = 0.5$ (three reviews and one missed fraud) and \$500 at $\tau = 0.85$, against a minimum of \$250 at $\tau = 0.25$, where five transactions are flagged and none is missed. The default threshold is 29% more expensive than the best, and the optimum is no accident of this sample: flagging pays whenever the expected loss avoided exceeds the review cost, $p \times \$200 > \$50$, so the cost-minimizing threshold is the ratio $\$50/\$200 = 0.25$. A threshold of 0.5 is correct only when a missed fraud costs exactly twice a review. Section 15.6.1 runs the calculation at the scale of a real alert queue.

15.2.5 Class Imbalance: Reweighting and Resampling

The examples so far used ordinary logistic regression to keep the statistical issue, extreme class imbalance, transparent. Production systems add more expressive models and dedicated imbalance techniques, which solve different parts of the problem and are often combined.

Cost-Sensitive Learning: Class Weighting

The most direct fix reweights the training loss so that a missed fraud case (a false negative) counts for more than a false positive. Refitting Section 15.2.3's logistic regression on the ten transactions of Table 15.2, with each class weighted inversely to its frequency,

$$\text{Weight (legitimate)} = 10/(2 \times 6) \approx 0.83, \qquad \text{Weight (fraud)} = 10/(2 \times 4) = 1.25,$$

shifts the fitted coefficients to

$$\hat{\beta}_0 \approx -2.25, \qquad \hat{\beta}_1 \approx 0.86, \qquad \hat{\beta}_2 \approx 0.13, \qquad \hat{\beta}_3 \approx 0.85,$$

and raises transaction 9's predicted probability from 33% to about 41.7%, still short of 0.5. At the unchanged 0.5 threshold, class weighting alone makes this small sample's confusion matrix worse: transaction 4 crosses into a false positive, giving precision 0.75, recall unchanged at 0.75, and $F_1 \approx 0.75$, below the original model's 0.857. Class weighting reshapes the decision boundary, but it does not guarantee that a missed case crosses the existing threshold, and on a sample this small its effect is noisy.

Lowering the threshold to 0.40 alongside the reweighted coefficients (compare Table 15.3) does catch transaction 9 (now at 41.7%) while still flagging transaction 4, giving precision 0.80, recall 1.00, and $F_1 \approx 0.889$,

Wulin.Suo@Queensu.ca

the same operating point the unweighted model reached at a threshold of 0.30. For a linear model, class weighting and threshold selection move the decision boundary in closely related ways, so a team tuning one should revisit the other.

Resampling: Undersampling and SMOTE

A second family of techniques changes the training data rather than the loss function. Random undersampling discards a random subset of majority-class (legitimate) observations until the training set is more balanced: cheap, but wasteful, since it throws away information about legitimate behavior that helps the model avoid false positives. The Synthetic Minority Oversampling Technique (SMOTE) instead generates synthetic fraud examples: for each real fraud case, it finds its nearest neighbors among other fraud cases (Chapter 6's k-nearest-neighbors distance calculation, used here to construct training points rather than to classify) and creates synthetic points along the line segments connecting them. This adds minority-class examples without discarding legitimate data, but a synthetic point interpolated between two fraud cases is not a real transaction. If the two cases do not represent a coherent fraud pattern, SMOTE can teach a model a feature combination that corresponds to no real fraud behavior, a subtler version of the overfitting risk Chapter 6 introduced.

Applying SMOTE to Table 15.2's four fraud cases (transactions 1, 3, 6, and 9), asking for two synthetic points with $k = 3$ neighbors, produces:

$$\begin{aligned}
&\text{Synthetic point 1:} \quad (1.836,\ 2.763,\ 1), \quad 39.7\% \text{ from transaction 9's } (1.2, 0.3, 1) \\
&\qquad\qquad\qquad\qquad\ \text{toward transaction 3's } (2.8, 6.5, 1), \\
&\text{Synthetic point 2:} \quad (3.263,\ 8.654,\ 1), \quad 54.5\% \text{ from transaction 1's } (3.1, 8.0, 1) \\
&\qquad\qquad\qquad\qquad\ \text{toward transaction 6's } (3.4, 9.2, 1),
\end{aligned}$$

verified by solving for each point's interpolation fraction λ along the amount-z-score axis and confirming it reproduces the distance coordinate exactly. Neither point is an observed transaction: both fall between similar fraud cases here, but SMOTE cannot know whether that holds for every pair it selects on a larger, messier dataset.

15.2.6 More Expressive Models: Ensembles and Unsupervised Anomaly Detection

Logistic regression's score is additive in its features, which is part of why Section 15.2.3's coefficients are easy to interpret, but real fraud often depends on *combinations* of features that an additive model can only approximate. Chapter 6's tree-based ensembles, Random Forest and gradient boosting, split the feature space recursively and can represent a rule like "flag this transaction only if the amount is unusual *and* it occurs at an unusual location *and* at an unusual time, but not if only one of these is true," a nonlinear interaction a single logistic coefficient vector cannot express. This flexibility does not solve class imbalance (a gradient-boosted model trained on the same skewed data shows the same base-rate problem unless combined with class weighting or resampling), and it costs interpretability, which Section 15.6.2 returns to as a regulatory constraint.

Since ten transactions are too few to show an ensemble advantage, the companion notebook simulates 5,000 transactions with a three-way interaction built into the fraud-generating process: amount, distance, and lateness each contribute modestly on their own, but fraud probability jumps only when all three are unusual at once. It then evaluates a logistic regression and a gradient-boosted ensemble, with matched class weighting and no information about the interaction structure, on a held-out test set using precision-recall AUC, the metric argued for below.

Logistic regression, unable to represent the three-way interaction, reaches a PR-AUC of 0.2847; gradient boosting, which can split on combinations of features, reaches 0.3795, a 33.3% relative improvement, with feature importances of 0.408 (amount), 0.490 (distance), and 0.102 (lateness) confirming it uses all three features jointly. This is a real improvement, not the in-sample perfection Chapter 9's Gini/AUC validation example warned against, because it is measured on data the ensemble never saw.

A fundamentally different approach abandons supervised labels. Isolation Forest and autoencoder-based anomaly detection learn what *normal* behavior looks like from the (overwhelmingly legitimate) data itself,

Wulin.Suo@Queensu.ca

without being told which cases were confirmed fraud, and flag observations that are isolated or poorly reconstructed relative to that norm. This matters because confirmed-fraud labels can take weeks to arrive (Section 15.4.2), and adversarial adaptation means a labeled training set always describes yesterday's fraud. An unsupervised detector needs no label to flag a new pattern, but its flag only says "this looks unusual," not "this is fraud," so it typically has even lower precision than the supervised models and is best used as one more signal alongside them, not a replacement.

Fitting an Isolation Forest to the same simulated training transactions, without the fraud labels, and scoring the held-out transactions confirms this works in practice: the 34 held-out fraud cases receive a mean anomaly score of 0.5525 versus 0.4658 for legitimate transactions, and the unsupervised score alone achieves a ROC-AUC of 0.7428 against labels it never saw, modest but real discriminatory power obtained without a single confirmed fraud label.

Evaluating Models Under Imbalance: Precision-Recall AUC, Not ROC-AUC

The receiver operating characteristic (ROC) curve plots the true-positive rate against the false-positive rate across every threshold, and its area (ROC-AUC) is a common summary of a classifier's ranking ability. Under severe class imbalance, ROC-AUC can look deceptively good: because the false-positive rate is a share of an enormous legitimate population, even a model that ranks fraud only modestly well can show a high ROC-AUC, the base-rate distortion of Section 15.2.2. The precision-recall (PR) curve and its area (PR-AUC) avoid this blind spot, since precision is computed against the *predicted* positives rather than the huge negative population, which makes PR-AUC the more informative summary for rare events.

Figure 15.1 draws both curves for the two simulated models on the same 1,000 held-out transactions, 34 of them fraudulent, and the two summaries disagree. ROC-AUC prefers logistic regression, 0.7895 against 0.7316 for gradient boosting; PR-AUC reverses the order, 0.3795 against 0.2847. The metrics look in different places. Among the 20 transactions each model ranks highest, gradient boosting finds 14 frauds and logistic regression 10, a precision of 0.70 against 0.50; that is the top of an alert queue, where a bank operates, and it is where the precision-recall curve puts its weight. Logistic regression earns its ROC advantage far down the ranking: at a 40% false-positive rate it finds 26 of the 34 frauds against gradient boosting's 22, but only by flagging 386 legitimate transactions, a precision near 6% that no fraud team would accept. ROC-AUC weights every false-positive rate equally, so it rewards the ordering of transactions no one will review, while the operating region, the first percent or two of false positives, is a thin sliver at the plot's left edge. The right-hand panel also shows the baseline the ROC plot hides: a no-skill classifier sits on the diagonal in ROC space, a respectable-looking half of the area, but at a precision of 0.034, the base rate, in precision-recall space.

15.3 Two Fraud Types That Defeat Transaction-Level Scoring

The preceding part assumed a fraudulent transaction is, in principle, distinguishable from a legitimate one given enough features and a good enough model. Two of the fastest-growing categories of fraud violate that assumption, each in a different way, and both show where the machinery developed above stops being sufficient.

15.3.1 Synthetic Identity Fraud: The Case With No Labels

A synthetic identity is not a stolen one. The fraudster assembles a person who does not exist, typically pairing a real but unused identity number (often a child's, a recent immigrant's, or a deceased person's) with a fabricated name, date of birth, and address. Because no real person owns the composite, no one ever notices it, disputes a charge, or files a report.

That fact changes the detection problem. The account is then *cultivated*: it takes a small credit line, pays every bill on time, and is rewarded with progressively higher limits, as a creditworthy new customer would be. Suppose four tradelines are grown from a \$500 to a \$10,000 limit over eighteen months, with about \$120 a month paid across all of them,

$$\text{Paid in} = 18 \times \$120 = \$2{,}160, \qquad \text{Exposure at bust-out} = 4 \times \$10{,}000 = \$40{,}000,$$

Wulin.Suo@Queensu.ca

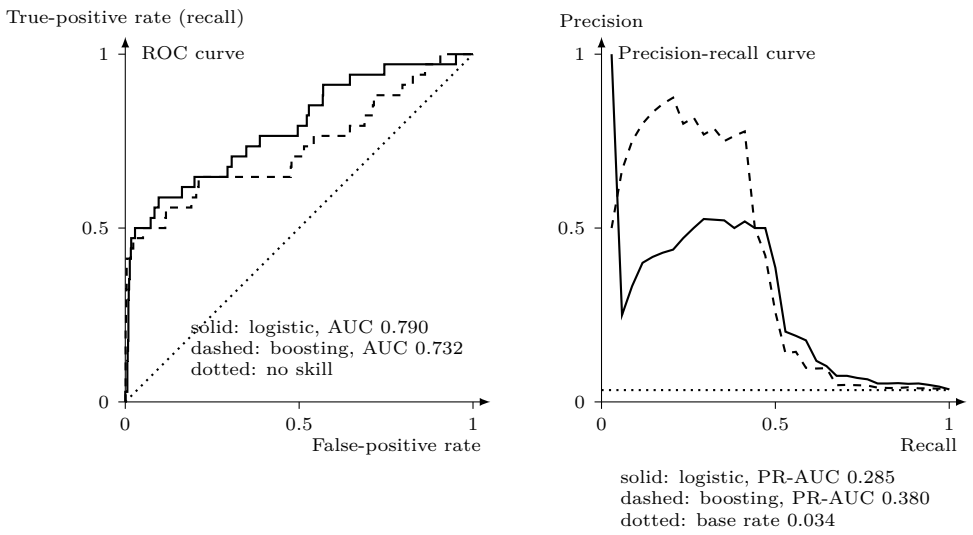


Figure 15.1: ROC and precision-recall curves for the logistic regression and gradient-boosting fraud models on 1,000 held-out transactions with 34 frauds. ROC-AUC ranks logistic regression first; PR-AUC, which weights the top of the alert queue, ranks gradient boosting first.

so the "bust-out," drawing every line to its limit and disappearing, nets roughly $37,840 on an outlay of $2,160, a return of about eighteen times. The behavioral risk score of Section 15.4.1 would have said throughout: velocity normal, geography stable, activity consistent with the profile declared at onboarding. Every input this chapter has built points the wrong way, because during cultivation the account is a model customer. The score spikes only at the bust-out, when the money is gone.

The deeper problem is the labels. A bust-out is usually written off as credit loss, since there is no victim and no obvious moment of theft. If 5% of a lender's 100,000 annual charge-offs are synthetic identities, then 5,000 fraud cases a year sit in the training data labeled "default," and a supervised fraud model never learns them. This is the structural blindness of Chapter 9's reject inference, in a more damaging form: the measured fraud base rate is understated, so every precision figure computed against those labels is optimistic by an unknown margin.

It is also the strongest argument in this chapter for the unsupervised methods of Section 15.2.6. An Isolation Forest or autoencoder needs no confirmed fraud label, and the cultivation pattern (several thin-file accounts sharing an address, device fingerprint, or phone number, all maturing on similar trajectories) is the kind of structure an anomaly detector or the network analysis of Section 15.5.1 can surface without one.

15.3.2 Authorized Push Payment Fraud: The Case Where Every Feature Is Legitimate

In an authorized push payment scam the customer is deceived, by a fake invoice, an impersonated bank official, or a romance or investment pretext, into sending the money themselves. The credentials are correct, the device is the customer's usual one, the authentication succeeds, and the instruction is genuine. This is transaction 9 of Table 15.2 carried to its limit: a payment indistinguishable from a legitimate one on every feature a transaction-level model observes, because it *is* a legitimate instruction from the account holder.

This was long treated as a customer-education problem, since a bank that correctly executed an authorized instruction had done nothing wrong. Mandatory reimbursement regimes, most prominently the United Kingdom's, which since 7 October 2024 has required the sending and receiving firms to split reimbursement of qualifying scam losses equally, moved the loss onto the bank's books and made it an expected-cost calculation.

Wulin.Suo@Queensu.ca

The economics then become favorable quickly. Suppose a bank processes 500,000 faster payments a month, 0.02% of which are scams, at an average loss of $4,200, and that it bears half of the reimbursement,

$$\text{Scams} = 0.0002 \times 500{,}000 = 100, \qquad \text{Bank's exposure} = 100 \times \$4{,}200 \times 0.5 = \$210{,}000 \text{ per month.}$$

A model operating at 60% recall and 5% precision, poor by this chapter's earlier standards, catches 60 scams while raising $60/0.05 = 1{,}200$ alerts. At 15 minutes per intervention (a call to the customer before the payment is released) and a fully loaded analyst cost of $45 an hour (the rule of thumb Section 15.6.1 also uses),

$$\text{Cost} = \frac{1{,}200 \times 15}{60} \times \$45 = \$13{,}500, \qquad \text{Loss prevented} = 60 \times \$4{,}200 \times 0.5 = \$126{,}000,$$

a net benefit of $112,500 a month, or $1.35 million a year. The break-even recall is 6.43%: the model pays for itself once it stops six or seven of the hundred monthly scams, a far more forgiving operating point than anything in Section 15.6.1, because a missed case now costs the bank $2,100 rather than nothing.

Because the instruction itself carries no signal, detection has to look elsewhere: at deviation from the customer's own payment history (a first-time payee, an unusual amount, an account that has never sent a faster payment), at context such as a payment initiated during or right after a long inbound call, and above all at the *receiving* side, where mule accounts show the short-dwell, high-pass-through pattern of Section 15.5.1's layering example. The victim's transaction looks normal; the beneficiary's does not.

Worked Example 15.3: Scoring a Real-Time Payment Under a Latency Budget

The calculation above treated the model as a recall and a precision. On an instant-payment rail it must return a decision while the customer waits, and the institutions around it have changed quickly. In the first eighteen months of the United Kingdom's regime, from 7 October 2024 to 31 March 2026, the Payment Systems Regulator reports that 88% of the money lost to APP scams, £316 million, was returned to victims; reimbursement is capped at £85,000 per claim, and the sending firm may charge an excess of up to £100 (PSR 2026). Firms received a matching tool: since 30 October 2024 a UK payment firm may delay an outbound payment by up to four business days when it has reasonable grounds to suspect APP fraud (FCA 2024), which makes a *hold* (a payment paused while the customer is called) a lawful alternative to releasing or refusing it. The problem is spreading with the rails. The Federal Reserve's FedNow service went live in July 2023 and raised its per-payment limit to $10 million in November 2025 (Federal Reserve Financial Services 2025), and the European Union's Instant Payments Regulation requires euro-area payment firms to check, from October 2025, that the payee's name matches the account before a transfer is sent (European Parliament and Council of the European Union 2024c). In the United States, by contrast, Regulation E's consumer protections are built around *unauthorized* transfers, so a payment a deceived customer authorized generally falls outside them, and who bears the loss is set by network rules and each bank's policy, not a federal reimbursement mandate.

Consider one payment, scored at the moment the customer presses "send." Because the evidence arrives as separate signals, the natural first model is the Naive Bayes form of Chapter 1's sequential Bayesian updating: if the signals are independent given the payment's true class, the posterior log-odds is the prior log-odds plus one log-likelihood ratio per signal. Suppose that a new payee appears in 85% of scams but 10% of legitimate payments, a phone call in progress during the session in 60% against 3%, a payee account opened within the last 30 days in 70% against 2%, and a payee account that forwards more than 90% of what it receives within 24 hours in 75% against 3%, and that the amount's z-score against the customer's history is distributed $N(1.5, 1)$ for scams and $N(0, 1)$ for legitimate payments, so that its likelihood ratio is $e^{1.5z - 1.125}$. The payment goes to a new payee, its amount is ordinary ($z = 0.5$), and no call is in progress. From the sender's side alone, starting from the 0.02% scam rate,

$$\begin{aligned} \text{log-odds} &= \ln \tfrac{0.0002}{0.9998} + \ln \tfrac{0.85}{0.10} + (0.75 - 1.125) + \ln \tfrac{0.40}{0.97} \\ &= -8.517 + 2.140 - 0.375 - 0.886 = -7.638, \qquad \Pr(\text{scam}) = 0.048\%, \end{aligned}$$

about 2.4 times the prior, but far too low to act on. The payee's account, however, was opened nine days ago and forwards nearly everything it receives, adding $\ln 35 = 3.555$ and $\ln 25 = 3.219$:

$$\text{log-odds} = -7.638 + 3.555 + 3.219 = -0.864, \qquad \Pr(\text{scam}) = 29.66\%.$$

Wulin.Suo@Queensu.ca

The two halves of the evidence disagree by a factor of more than 600: everything suspicious about this payment is on the receiving side.

Whether to hold follows from the same cost comparison as Equation (15.2). A hold costs a 15-minute call at \$45 an hour plus an assumed \$5 of customer friction, \$16.25 in all, and the call breaks the scam in an assumed 70% of cases, so with the bank bearing half of a \$4,200 loss a hold pays when $p \times 0.70 \times \$2{,}100 > \$16.25$, that is, above

$$p^* = \frac{\$16.25}{\$1{,}470} = 0.01105.$$

The sender-side score implies an expected saving of $0.00048 \times \$1{,}470 = \$0.71$, so the payment is released; the full score implies \$436.00, so it is held. The threshold of about 1.1% is the reason a model with 5% precision paid for itself above: under mandatory reimbursement a bank should rationally hold payments it believes are almost certainly genuine.

The payee-side signals are also the slow ones. They live at the receiving firm or a shared intelligence service, and fetching them costs a network round trip inside the decision budget. Suppose 220 milliseconds of the budget remain once the sender-side model has run, and the lookup's latency is lognormal with a median of 80 ms and a log standard deviation of 0.6. The lookup then misses the budget with probability

$$\Pr\left(Z > \frac{\ln(220/80)}{0.6} = 1.686\right) = 0.0459,$$

which rises to 0.1240 with 160 ms left and 0.2496 with 120 ms, and falls to 0.0138 with 300 ms. Table 15.4 shows what the system should do when the answer is late. The companion notebook simulates 5,000,000 payments containing 1,000 scams, scores each with the model above, and rescales to the earlier example's 500,000 payments and 100 scams a month, counting \$16.25 per hold, \$2,100 per scam released, and 30% of \$2,100 for each scam held but not stopped.

Policy	Holds	Recall	Precision	Monthly cost (\$)
Release every payment	0	0.000	–	210,000
Sender-side score only	1,089.6	0.519	0.0476	151,413
Full score, lookup always on time	659.8	0.856	0.1297	94,890
Full score, on timeout use sender score	675.6	0.832	0.1231	98,674
Full score, on timeout hold	23,557.5	0.860	0.0037	466,389

Table 15.4: Monthly outcomes of five holding policies for 500,000 instant payments containing 100 scams, at the hold threshold $p^* = 0.01105$ and a 4.59% chance that the payee lookup misses the latency budget

Three lessons follow. First, the receiving-side signals improve recall and precision at once, from 51.9% to 85.6% and from 4.8% to 13.0%, while *reducing* holds from about 1,090 to 660 a month, and they cut the monthly cost by \$56,523; on the sender side alone the scam and the genuine payment look alike. Second, when the lookup is late the right fallback is the next-best score, which costs only \$3,784 a month against a system that is never late. Third, the instinct to fail safe by holding whatever cannot be scored in time is ruinous: 4.59% of 500,000 payments is 22,950 extra holds, and the policy costs more than twice as much as releasing everything. The latency budget is therefore a modeling parameter, not an engineering detail: it decides how often the model's best evidence is available. Two cautions apply to the numbers. The likelihood ratios are assumptions, and a young account and a high pass-through rate are both symptoms of a mule, so treating them as independent overstates the evidence; the 29.66% is too high, and a fitted logistic regression, which learns that correlation, would temper it. And the 70% stop rate is the least certain parameter, since a customer coached by a scammer will often insist that the payment is genuine.

15.3.3 Generative AI as an Accelerant: Deepfakes and Forged Identities

Generative AI has handed both preceding fraud types a cheap accelerant. Image generators can produce a convincing face for a nonexistent person, or alter a real identity document, and pair it with stolen or

Wulin.Suo@Queensu.ca

fabricated data: the synthetic identity of Section 15.3.1, now with a photograph and a document that pass the remote identity verification know-your-customer (KYC) rules require at account opening. Voice cloning and real-time video impersonation give an authorized push payment scam what it used to lack, a familiar face and voice. In the best-documented case, reported by Hong Kong police in February 2024, a finance employee in the Hong Kong office of a multinational, later confirmed as the British engineering firm Arup, joined a video conference in which the chief financial officer and several colleagues were all deepfakes, and made 15 transfers totaling HK$200 million, about US$25 million, to accounts the fraudsters controlled (Chen and Magramo 2024; Magramo 2024). Every payment was authorized by a real employee on the firm's own systems: Section 15.3.2's pattern at corporate scale.

Regulators have taken note. On 13 November 2024 the U.S. Financial Crimes Enforcement Network (FinCEN) issued alert FIN-2024-Alert004, reporting a rise since 2023 in suspicious activity reports describing suspected deepfake media, above all falsified identity documents used to circumvent verification, and asking institutions to tag such reports with the key term "FIN-2024-DEEPFAKEFRAUD" (FinCEN 2024a). Its red flags read like a feature list for a detection model: a photo inconsistent with the customer's other identifying information, identity documents inconsistent with each other, a third-party webcam plugin or a sudden request to change communication method during a live verification check, a refusal to use multi-factor authentication, and device or location data that do not match the documents.

The industry response has three layers, and only the first inspects the media itself. *Liveness detection* prompts the applicant to respond in a live audio or video session and looks for signs of synthesis or injected video; it is an arms race, and FinCEN's alert notes that criminals may answer live prompts with tools that generate synthetic audio and video, although the responses may still reveal inconsistencies. *Out-of-band verification*, confirming an instruction through an independently known channel (a call back to a number already on file, not one supplied in the request), defeats impersonation however good the fake; it would have stopped the Hong Kong transfers. *Behavioral and device signals*, the account-level features of Section 15.4.1 and the receiving-side mule detection above, observe what the fraudster does rather than looks like, and so are the layer least eroded by better generators. The lesson generalizes: once the artifact a control inspects can be forged at near-zero cost, detection has to move to signals that remain expensive to fake.

15.4 Risk Tiering and Survival Analysis

Beyond a single score, this part ranks customers by behavior and models time-to-detection.

15.4.1 Behavioral Risk Frameworks and Customer Risk Tiering

Chapter 9's credit-scoring model mapped a predicted default probability into letter-grade ratings; in fraud and AML, institutions apply an analogous mapping, customer risk tiering, built from behavioral features accumulated over the whole account relationship rather than a single transaction.

Typical inputs include the account's transaction velocity (how this month's frequency and size of transactions compare to its own trailing average, an account-wide analogue of Section 15.2.3's amount z-score), geographic diversity of counterparties, newly added payees, and, for anti-money laundering (Section 15.5.1), whether the occupation and expected volume the customer declared at onboarding under Know-Your-Customer (KYC) requirements are consistent with observed activity.

Table 15.5 shows an illustrative scheme built from a composite behavioral score on a 0-100 scale, combining such features with weights calibrated from the institution's own fraud and AML case data, like Chapter 9's rating buckets.

Composite score	Risk tier	Typical monitoring response
0–29	Low	Standard, periodic account review
30–59	Medium	Increased transaction monitoring sensitivity
60–84	High	Manual review of new activity; tighter velocity limits
85–100	Enhanced due diligence	Senior compliance review; may restrict or close account

Table 15.5: An illustrative customer risk-tiering scheme

Wulin.Suo@Queensu.ca

A concrete composite score makes Table 15.5 usable. Suppose an institution combines three behavioral sub-scores, each normalized to a 0-100 scale: a transaction-velocity sub-score (how far this month's activity deviates from the account's trailing average), a geographic-diversity sub-score (how unusual the mix of counterparty locations is), and a KYC-consistency sub-score (how far observed activity deviates from what the customer declared at onboarding), weighted 40%, 35%, and 25% based on the institution's backtested case outcomes.

For an account with a velocity sub-score of 70 (volume well above its recent average), a geographic-diversity sub-score of 50 (a moderate increase in counterparty locations), and a KYC-consistency sub-score of 90 (activity far exceeding what was declared), the composite score is

$$0.40(70) + 0.35(50) + 0.25(90) = 28.0 + 17.5 + 22.5 = 68.0,$$

placing the account in the High tier of Table 15.5, triggering manual review and tighter velocity limits, even though no single sub-score (velocity at 70, geographic diversity at 50) would necessarily have triggered that response alone.

This is the point of a composite score: the KYC-consistency mismatch, the largest contributor at 22.5 of the 68.0 total, combines with two moderate signals to cross a review threshold that no single signal reached, the accumulating, multi-signal evidence of Chapter 1's sequential Bayesian updating example.

The weights are written here as though a committee chose them, and in many institutions one did. But calibrating them from historical case data is supervised learning: given a labeled history of which accounts eventually warranted a report, the weights are fitted coefficients from the logistic regression of Section 15.2.3, and the tiers discretize its fitted score, as Chapter 9's letter grades discretize a predicted default probability.

Tiering changes what the system optimizes for: rather than asking whether *this* transaction looks fraudulent, it asks whether *this account* has become risky enough to warrant tighter monitoring of everything it does. That is the right framework for a case like transaction 9 of Table 15.2, unremarkable alone but possibly the first sign of a compromised account whose score has begun drifting upward.

Regulators (Section 15.5.1) generally require such ongoing, risk-based due diligence rather than a one-time onboarding check, because a customer's risk, like credit risk in Chapter 9's rating-transition framework, evolves.

15.4.2 Survival Models: Time to Detection and Account Risk Over Time

Every classification model in this book, including Section 15.2.3's logistic regression, answers a yes-or-no question at a single point in time. A new question, and one institutions care about operationally, is how long a flagged account survives before fraud is confirmed and the account is closed. The answer helps a compliance team prioritize alerts and a fraud team estimate how much exposure a compromised account accumulates before detection.

Survival analysis, developed in biostatistics to study time until death or disease recurrence, answers this kind of question for data in which the outcome, here fraud confirmation, has not yet happened for every observation. The key complication, which ordinary regression cannot handle, is censoring: some accounts are closed for unrelated reasons (voluntary closure, a non-fraud risk exit) or are still open when the data are pulled, so their time to fraud confirmation is known only to be *at least* the observed time.

Dropping censored observations, or treating them as if fraud were confirmed at the moment of censoring, both bias the estimate; the Kaplan-Meier estimator instead uses every observation up until the point it is censored. The estimator computes the survival function, the probability an account remains unconfirmed (open, or closed for reasons unrelated to fraud) beyond time t, as a product of conditional survival probabilities at each observed event time,

$$\hat{S}(t) = \prod_{t_i \le t} \left(1 - \frac{d_i}{n_i}\right), \tag{15.3}$$

where n_i is the number of accounts still under observation (not yet closed or censored) just before time t_i, and d_i is the number of confirmed-fraud closures at time t_i. This book's companion technical note on the Kaplan-Meier estimator shows that this product-limit formula is not an ad hoc device but the exact nonparametric maximum-likelihood estimator of the survival function once right censoring is written into the likelihood, which is also why the curve steps down only at event times, never at censoring times.

Wulin.Suo@Queensu.ca

15.4.3 Worked Example 15.4: Kaplan-Meier Survival of Flagged Accounts

Suppose a bank's fraud team tracks 12 accounts flagged by Section 15.4.1's risk-tiering system, recording the days until a confirmed fraud closure (an event) or until an unrelated closure or the end of the observation window (censored, marked +): 5, 8, 8^+, 12, 15, 15^+, 20, 24^+, 30, 30^+, 35, 40^+. Table 15.6 works through the Kaplan-Meier calculation at each day on which a confirmed fraud closure occurred.

Day t_i	At risk n_i	Fraud closures d_i	Factor $(1 - d_i/n_i)$	$\hat{S}(t_i)$
5	12	1	0.9167	0.9167
8	11	1	0.9091	0.8333
12	9	1	0.8889	0.7407
15	8	1	0.8750	0.6481
20	6	1	0.8333	0.5401
30	4	1	0.7500	0.4051
35	2	1	0.5000	0.2025

Table 15.6: Kaplan-Meier survival calculation for 12 flagged accounts

Two features of Table 15.6 stand out. First, the number at risk, n_i, drops not only at a fraud closure but also whenever a censored observation passes, which is why n_i falls from 12 to 9 between day 5 and day 12 although only two closures occurred in between: the account censored on day 8 also leaves the risk set, without contributing a d_i term.

This is how censored observations contribute information: they stay in the risk set for as long as they are observed, without being counted as either a confirmed event or a survivor beyond their last observed day.

Second, the estimated probability that a flagged account remains unconfirmed beyond 35 days is only about 20.3%. Roughly 80% of accounts eventually confirmed as fraud are therefore confirmed within a little over a month of being flagged. A compliance team can use that number to set service-level targets for completing an investigation, and to recognize that an account still unresolved well beyond that horizon is atypical, an idea developed next.

Figure 15.2 draws the estimate as a step function, flat between events and dropping only on the seven days a fraud closure occurred; the five censored accounts appear as tick marks where they leave the risk set without moving the curve (the three censored on event days 8, 15, and 30 leave just after that day's drop). The steps grow as the curve moves right, from 1/12 of the remaining probability at day 5 to half of it at day 35, because each event removes one account from a shrinking risk set; the curve first falls to one half at day 30, the estimated median time to confirmation. The dashed band is the 95% interval derived in the next subsection.

15.4.4 How Precise Is $\hat{S}(t)$? Greenwood's Formula

A survival estimate like $\hat{S}(35) \approx 0.2025$ is half the story without a sense of how precisely 12 accounts pin it down.

Greenwood's formula gives the standard error of the Kaplan-Meier estimator at any time t,

$$\widehat{\mathrm{Var}}[\hat{S}(t)] = \hat{S}(t)^2 \sum_{t_i \le t} \frac{d_i}{n_i(n_i - d_i)}, \tag{15.4}$$

built from the at-risk counts n_i and event counts d_i already in Table 15.6. Applying this at $t = 20$, where $\hat{S}(20) \approx 0.5401$, the summed term $\sum d_i/(n_i(n_i - d_i))$ across the five event times up to and including day 20 is approximately 0.08175, giving $\widehat{\mathrm{Var}}[\hat{S}(20)] \approx 0.5401^2(0.08175) \approx 0.02385$ and a standard error of SE ≈ 0.154, so a normal-approximation 95% confidence interval is $0.540 \pm 1.96(0.154) \approx (0.237, 0.843)$, wide enough to span nearly the entire plausible range.

At $t = 35$, the final event time, $\hat{S}(35) \approx 0.2025$ with SE ≈ 0.165, giving a 95% interval of approximately (0.000, 0.526) once the lower bound is truncated at zero (a known limitation of the normal approximation

Wulin.Suo@Queensu.ca

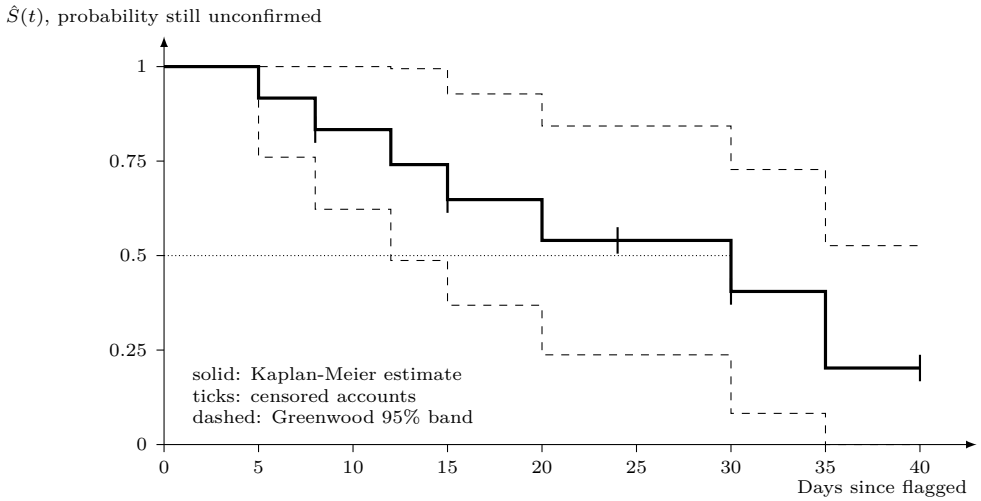


Figure 15.2: Kaplan-Meier survival curve for the 12 flagged accounts of Table 15.6, with censoring marks and the Greenwood 95% band

applied to a probability): the point estimate of roughly 20% is consistent with a 35-day survival probability anywhere from essentially zero to over 50%.

The dashed band of Figure 15.2 applies the same calculation at every event day, widening as the risk set shrinks. This is the small-sample caution of Chapter 7's AR(1) confidence interval and Chapter 9's collateral-coverage regression: with 12 accounts and 7 confirmed closures, the curve's later points carry large uncertainty however precise their four decimal places look.

The 80%-confirmed-within-35-days figure is a planning number from a small pilot sample, not a population parameter, and its interval will narrow as outcomes accumulate.

This is where the survival curve earns its prioritization role, since what orders an alert queue is a *conditional* probability read off $\hat{S}(t)$. A freshly flagged account is confirmed within its first fifteen days with probability $1 - \hat{S}(15) = 0.352$, whereas an account still unresolved at day 20 is confirmed by day 35 with probability $1 - \hat{S}(35)/\hat{S}(20) = 1 - 0.2025/0.5401 = 0.625$, nearly twice as likely to convert over the coming fortnight as the fresh flag is over its first. An investigator should therefore open the aging case before the newest one, the opposite of the instinct to work a queue newest-first, and this follows from Table 15.6 without any additional data.

Kaplan-Meier is nonparametric and uses no covariates, so it can say *when* flagged accounts are confirmed but not *which* ones. The learning extensions supply the missing half. Cox proportional hazards multiplies a baseline hazard by a function of an account's features, and survival forests and gradient-boosted survival models drop the proportionality assumption, so the same censored data yield a per-account predicted time to confirmation, which is what an alert-ranking system consumes. Chapter 9's discrete-time hazard formulation is the credit-risk counterpart of the same idea.

15.5 Money Laundering and Sanctions

This part turns to laundering typologies, sanctions screening, and detection shared across institutions.

15.5.1 Anti-Money Laundering: Structuring, Monitoring, and Regulation

Money laundering, disguising the origin of criminal funds so that they appear to come from a legitimate source, differs from transaction fraud: the funds are often not stolen from the institution, and the concealed crime frequently occurred entirely outside the financial system (drug trafficking, corruption, sanctions evasion) before the money reached a bank.

Wulin.Suo@Queensu.ca

In the United States, the Bank Secrecy Act requires banks to file a Currency Transaction Report (CTR) for any cash transaction exceeding $10,000, and a Suspicious Activity Report (SAR) for any transaction pattern, regardless of size, that appears designed to evade reporting requirements or otherwise suggests criminal activity; equivalent frameworks exist in most jurisdictions under the international standards set by the Financial Action Task Force (FATF). The European Union is replacing its patchwork of nationally transposed directives with a 2024 legislative package, published in the Official Journal on 19 June 2024: a directly applicable AML Regulation that applies from 10 July 2027, a sixth AML Directive, and a new EU Anti-Money Laundering Authority (AMLA) seated in Frankfurt, which began operating in July 2025 and, as of October 2026, is expected to select around 40 high-risk obliged entities during 2027 for direct supervision from 2028 (European Parliament and Council of the European Union 2024b; European Parliament and Council of the European Union 2024a; AMLA 2025).

Worked Example 15.5: Detecting Structuring

Structuring (sometimes called "smurfing") is the practice of breaking a large cash transaction into several smaller ones, each below the CTR reporting threshold, to avoid triggering a report. Suppose a customer makes five cash deposits of $9,500 each over three days, each individually $500 below the $10,000 CTR threshold,

$$\text{Total deposited} = 5 \times \$9{,}500 = \$47{,}500,$$

an amount that would trigger a CTR as a single deposit but, spread across five deposits each under the threshold, triggers none. This is why AML monitoring cannot rely on Section 15.2.2's single-transaction rules: "flag any transaction over $10,000" is by construction blind to a pattern engineered to stay under it.

Structuring detection instead applies a rule across a rolling window, for example flagging any customer with three or more cash deposits within seven days each within $1,000 of the threshold, which the five $9,500 deposits would trigger even though no single deposit would. This is the account-level monitoring philosophy of Section 15.4.1: the signal is a pattern across transactions and time, not any single transaction.

15.5.2 Placement, Layering, and Integration: A Network Example

Structuring is the classic technique for the first of the three stages of laundering: *placement*, introducing cash into the financial system, the deposit pattern above. *Layering* follows: moving the placed funds through a chain of transactions and accounts, often across institutions and jurisdictions, to obscure their criminal origin.

Integration is the final stage, reintroducing the layered funds into the legitimate economy, for example through a real estate purchase or an investment in a business, after which they are practically indistinguishable from legitimately earned money.

Layering is where the network-based detection of Table 15.1 becomes necessary, because no single account's activity, viewed in isolation, looks suspicious. Suppose $500,000 in placed funds moves through four shell entities, Alpha, Beta, Gamma, and Delta, each receiving the funds and forwarding 98% of what it received onward within 24 hours (retaining 2%, nominally a service or consulting fee), a repeating pattern that Table 15.7 traces hop by hop.

Entity	Received	Forwarded (98%)	Retained (2%)
Alpha	$500,000.00	$490,000.00	$10,000.00
Beta	$490,000.00	$480,200.00	$9,800.00
Gamma	$480,200.00	$470,596.00	$9,604.00
Delta	$470,596.00	$461,184.08	$9,411.92

Table 15.7: A four-hop layering chain: funds received, forwarded, and retained at each shell entity

After four hops, $461,184.08, over 92% of the original placed amount, reaches its destination, with only $38,815.92 lost to retained "fees," having passed through four legally separate entities in under a week. Figure 15.3 draws this chain as the network it is, rather than as four individually unremarkable rows in a table.

Wulin.Suo@Queensu.ca

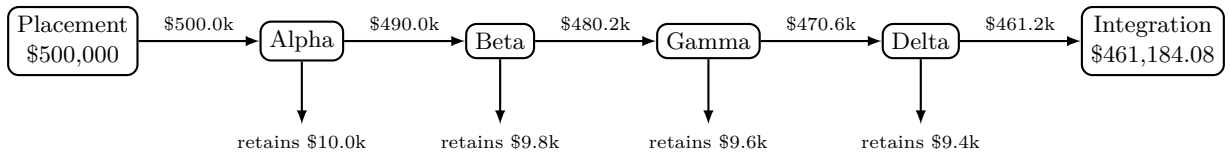


Figure 15.3: The four-hop layering chain of Table 15.7 as a network

No individual hop in Table 15.7 looks wrong; each is a plausible business payment of a plausible size. What is anomalous is the pattern viewed as a network, and each of its three features is a number rather than an impression: a pass-through ratio of exactly 0.98 at every hop, where a real operating business retains and redeploys most of what it receives; a dwell time under 24 hours, against the weeks a real payables cycle takes; and, at every interior node, exactly one party paying in and one party paid out, an in-degree and out-degree of one in the vocabulary of graphs, where a real supplier has many customers and many suppliers. Each has innocent explanations on its own. All three at once, repeated four hops in a row, do not.

That conjunction is a reason to look, not a conclusion. A factoring company, an escrow agent, or a freight forwarder shows high pass-through and short dwell as ordinary business, and a sole-source subcontractor does have one customer and one supplier; the network's shape alone cannot supply a reason to believe the money is criminal. Three further tests do that work. The first is *commercial substance*: a real intermediary has employees, premises, and counterparties outside this chain, whereas Alpha has a bank account, an incorporation date shortly before the flows began, and a registered address shared with dozens of unrelated entities. The second is *common control*: shared directors or beneficial owners, a shared address, telephone number, or device, or four accounts opened at one branch within days of each other, which turns four legally separate companies into one hand moving its own money and has no innocent reading. The third is the fee. Two percent of principal, identical at every hop, is a cut rather than a price, because a real service is charged for work performed and does not scale with the sum passing through. Only with such corroboration does an alert amount to the suspicion of criminal origin a Suspicious Activity Report requires; the graph by itself reports, as Section 15.2.6 said of any unsupervised flag, that something is unusual, not that it is a crime.

This is why anti-money-laundering systems increasingly supplement Section 15.4.1's per-account scoring with graph-based analysis of the relationships between accounts, since a layering scheme is constructed so that every individual node looks unremarkable.

As a learning problem, this is node and subgraph classification on the transaction graph. Community detection isolates tightly connected clusters that ordinary commerce does not produce, and graph neural networks, which Chapter 17 develops on this layering network, learn each account's representation from its neighbors' features as well as its own, scoring an account on the company it keeps. The supervision is weak, since confirmed laundering is rare and arrives late, so in practice these models rank leads for investigators rather than deciding anything.

Figure 15.4 places structuring and layering detection alongside the chapter's statistical and behavioral tools, showing how a monitoring system combines several signals before a case reaches a human investigator.



Figure 15.4: The fraud and AML monitoring pipeline

15.5.3 Ongoing Due Diligence and Adverse-Media Screening

AML regulation also imposes an ongoing due-diligence obligation that goes beyond fraud detection's largely reactive posture. Customer Due Diligence (CDD) at onboarding establishes a customer's expected activity profile (Section 15.4.1's KYC inputs); Enhanced Due Diligence (EDD) applies heightened scrutiny to higher-risk customers such as politically exposed persons or those in high-risk jurisdictions; and ongoing monitoring

Wulin.Suo@Queensu.ca

continuously compares behavior against that profile, the time-varying risk view Section 15.4.2's survival framework formalizes.

Two of these obligations are increasingly discharged with the language models of Chapter 16. Adverse-media screening, checking whether a customer appears in news reporting tied to fraud, corruption, or sanctions evasion, is a document-classification problem of the kind that chapter's Naive Bayes and embedding methods address, and it inherits this chapter's precision problem in acute form, since most name matches in news text are the wrong person. Drafting the suspicious-activity report itself, a structured account of what was observed and why it appears suspicious, is now commonly assisted by a large language model working from the alert's records. Both uses are governed rather than autonomous: a filed SAR is a legal statement by the institution, so the model proposes, an investigator signs, and the model is subject to the validation regime of Section 15.6.2.

15.5.4 Sanctions Screening: A Matching Problem, Not a Lookup

A bank must not move money for a party on a government sanctions list, and unlike almost every other obligation in this chapter, this one admits no risk appetite: in the United States liability under the Office of Foreign Assets Control (OFAC) regime is strict, so a bank that processes a prohibited payment has violated the law whether or not it knew, intended, or profited.

It is tempting to conclude that sanctions screening is a database lookup rather than a modeling problem. The legal *consequence* of a match is binary, but the *match itself* is not, and conflating the two is the most common way to misunderstand where the difficulty lies. A watchlist records a name in one romanization; a payment message carries whatever spelling, word order, punctuation, and truncation the originating bank's system produced. Nothing guarantees the two strings agree even when they name the same person, and nothing prevents two different people from having names that differ by a single letter.

The standard tool for comparing two strings that should match but do not is the edit (Levenshtein) distance $d(a, b)$, the minimum number of single-character insertions, deletions, or substitutions turning one string into the other, normalized into a similarity score in $[0, 1]$,

$$\mathrm{sim}(a, b) = 1 - \frac{d(a, b)}{\max(|a|, |b|)}. \tag{15.5}$$

Two short surnames show the mechanics. Turning `SMITH` into `SMYTH` requires one substitution, so $d = 1$ and $\mathrm{sim} = 1 - 1/5 = 0.800$; turning `SMITH` into `SCHMIDT` requires four edits against a longer target, giving $\mathrm{sim} = 1 - 4/7 \approx 0.429$. A screening engine flags any pair scoring above some threshold and sends it to an analyst.

Table 15.8 applies this to a watchlist entry, `Mohammed Al-Rashid`, against seven incoming counterparty names, four of which are the same individual and three of which are not. Scores are computed twice: after simple normalization (uppercase, punctuation stripped), and after also sorting the name's tokens alphabetically, a standard step to defeat surname-first ordering.

Counterparty name	Same person?	Normalized	Token-sorted
Mohammad Al-Rashid	yes	0.9412	0.9412
Mohamed Alrashid	yes	0.9412	0.9412
Muhammad Al-Rasheed	yes	0.7778	0.7778
Al-Rashid, Mohammed	yes	0.2941	1.0000
Mohammed Al-Rashad	no	0.9412	0.9412
Ahmed Al-Rashid	no	0.7647	0.2941
Mohammed Al-Karim	no	0.7647	0.7647

Table 15.8: Name-similarity scores against the watchlist entry `Mohammed Al-Rashid`

Token sorting earns its place: `Al-Rashid, Mohammed` is the same name written surname-first, and plain edit distance scores it 0.2941, below every non-match in the table, because reversing two words costs almost

Wulin.Suo@Queensu.ca

as many edits as rewriting the string. Sorting the tokens raises it to 1.0000. The same step pushes `Ahmed Al-Rashid`, a different person, down from 0.7647 to 0.2941, improving separation in both directions.

The result that matters is the one no preprocessing fixes. `Mohammed Al-Rashad`, a different individual whose family name differs from the target's by a single vowel, scores 0.9412, tying *exactly* with two of the four true matches, while the weakest true match, `Muhammad Al-Rasheed`, scores only 0.7778. The highest-scoring non-match outranks a true match, and no threshold separates the two classes: set the bar at 0.90 and the engine clears a sanctioned party while flagging an innocent one; set it at 0.75 and it catches everything by flagging almost everyone.

This is not a defect a better string distance would repair. A name alone does not identify a person, and two of these strings are too close for any function of the characters to tell apart. Production systems therefore treat the string score as a first-stage filter and resolve the remainder with secondary identifiers (date of birth, nationality, passport or national identity number, address), which is why the Know-Your-Customer records collected at onboarding (Section 15.4.1) double as screening inputs.

Here strict liability bites. Section 15.2.3's threshold choice was an economic trade-off; here one missed true match is a legal violation however favorable the arithmetic, so a bank cannot buy precision with a little less recall. It sets the threshold low enough to make a miss nearly impossible and absorbs the resulting false positives, which is what makes screening expensive.

Suppose a mid-sized bank screens 1,200,000 payment messages a month against consolidated watchlists, that 15 of them involve a sanctioned party, and that a threshold set for near-certain recall produces a 0.5% false-positive rate against the remaining traffic. Then

$$\text{Alerts} = 0.99(15) + 0.005(1{,}199{,}985) \approx 6{,}015, \qquad \text{Precision} = \frac{14.85}{6{,}015} \approx 0.00247,$$

so roughly 0.25% of screening alerts are true matches, against the 3.0% alert-to-SAR conversion Section 15.6.1 computes for transaction monitoring: a precision twelve times worse. At 10 minutes per screening alert (half an AML alert's time, since most are cleared on sight) and the same \$45 hourly cost used above,

$$\text{Analyst hours} = \frac{6{,}015 \times 10}{60} \approx 1{,}002, \qquad \text{Monthly cost} \approx 1{,}002 \times \$45 \approx \$45{,}100,$$

or roughly \$541,000 a year. This is a second alert queue, three-quarters the size of the 8,000-alert transaction-monitoring queue analyzed in Section 15.6.1, running at one-twelfth its precision, at every institution that moves money across borders. Supervised learning enters here because labels, scarce almost everywhere else in this chapter, are abundant in screening: every alert is adjudicated by an analyst, so a bank accumulates hundreds of thousands of match and no-match decisions a year. A classifier trained on those dispositions scores a candidate pair on far more than string distance: how common the name is in the relevant population, whether date of birth, nationality, and document number agree, whether the two spellings belong to the same transliteration family, and which kind of list entry was matched. Because it *ranks* rather than thresholds, it can hold recall fixed while pushing most false positives below the line at which the queue is worked.

If a reader takes one operational fact from this chapter, it should be this: in most banks, name screening, not transaction monitoring, is the largest single source of compliance alerts, and reducing its false-positive rate without touching recall is among the highest-value applications of machine learning in financial crime compliance.

15.5.5 Privacy-Preserving Collaboration: Federated Learning Across Institutions

The layering networks above expose a weakness of single-institution detection: a laundering scheme spreads its transactions across many banks so that no single bank sees enough of the pattern. The obvious remedy, pooling transaction data, is largely prohibited: bank-secrecy and data-protection law restricts sharing customer data, and no bank wants its records in a competitor's hands. In federated learning, institutions jointly train one shared model *without any of them transmitting its underlying data.* Each bank fits the current shared model to its own transactions and sends back only the parameter estimates, and a coordinator combines them, most simply by federated averaging (a sample-size-weighted mean), into the next version of the model.

Wulin.Suo@Queensu.ca

A hand-checkable round shows the mechanics. Suppose three banks jointly train the fraud-scoring logistic regression of Section 15.2.3 on a shared structuring-intensity feature, holding 2,000, 5,000, and 3,000 labeled transactions respectively. Their local coefficient estimates come back as 0.90, 1.30, and 1.10, the smallest bank's being the noisiest. Federated averaging weights each estimate by its sample share,

$$\hat{\beta}_{\text{fed}} = \frac{2{,}000 \times 0.90 + 5{,}000 \times 1.30 + 3{,}000 \times 1.10}{10{,}000} = \frac{11{,}600}{10{,}000} = 1.16,$$

against an unweighted mean of 1.10; the weighting lets the estimate lean toward the banks with the most evidence. The payoff is clearest from the smallest bank's seat: scoring a transaction with structuring intensity $x = 2$ (with intercept -3), its own local model flags it with probability $\sigma(-3 + 0.90 \times 2) \approx 0.2315$, while the federated model assigns $\sigma(-3 + 1.16 \times 2) \approx 0.3363$, a sharper detector obtained without a single customer record leaving any bank.

Model updates are not automatically harmless: an estimate computed from a small dataset can leak information about the records behind it, so production systems add differential privacy, calibrated random noise added to each update before transmission so that no observer can reliably infer whether any single transaction was in the data. Aggregation makes this affordable: perturbing the three local estimates by illustrative noise draws of $+0.04$, -0.02, and $+0.01$ changes the federated coefficient only from 1.16 to 1.161, because independent noise largely cancels in a weighted average while masking each contribution. The privacy-utility trade-off does bind (more noise, more privacy, a worse model), and federated systems raise new operational questions (a malicious participant could poison the shared model through its updates, the adversarial dynamic of Chapter 18's robustness discussion moved into the training loop). But the core bargain, most of the pooled-data detection gain with none of the pooled-data legal exposure, is why central banks have begun testing it. The BIS Innovation Hub's Project Aurora, concluded in May 2023, combined privacy-enhancing technologies, machine learning, and network analysis on a synthetic dataset of domestic and cross-border payments with embedded laundering patterns, and found this collaborative analysis more effective than today's siloed, rules-based monitoring: it detected up to three times more laundering in complex schemes and cut false positives by up to 80% (BIS Innovation Hub 2023). Those are proof-of-concept results on simulated data, not production evidence; as of October 2026 a second phase is still under way.

15.6 Economics and Governance

A detection model's value is decided by the cost of its false positives and by who is accountable for it. This part is deliberately the shortest in the chapter: detection methods earn their length because they can be taught with worked examples, whereas the failures that produce enforcement actions are organizational, and are better shown once, by the case vignette that follows.

15.6.1 Alert Economics and the Cost of Low Precision

Section 15.2.2 showed that one bank's fraud model can generate 19,600 flagged transactions in a day; AML monitoring systems, which layer structuring rules, behavioral risk scores, and sanctions screening on top of transaction scoring, typically generate a comparably large alert volume relative to the number that warrant a filed report.

Suppose a bank's combined monitoring system generates 8,000 alerts in a typical month, of which 240 are ultimately investigated and filed as SARs,

$$\text{Alert-to-SAR conversion rate} = \frac{240}{8{,}000} = 3.0\%,$$

well below Section 15.2.2's 24.2% fraud-flag precision, since each additional rule or model adds its own false-positive rate. If each alert takes an average of 20 minutes to review, including the 97% ultimately cleared as false positives,

$$\text{Analyst hours per month} = \frac{8{,}000 \times 20 \text{ minutes}}{60} \approx 2{,}667 \text{ hours},$$

Wulin.Suo@Queensu.ca

roughly 16 to 17 full-time analysts working a standard month just to clear this one system's queue. At a fully loaded analyst cost of $45 per hour (a common rule of thumb including salary, benefits, training, and overhead), this is a monthly labor cost of

$$2{,}666.67 \times \$45 \approx \$120{,}000,$$

or roughly $1.44 million annually to staff alert review for one system at one institution, before the cost of data infrastructure, the models, or the SAR-filing process. This dollar figure makes precision a direct cost-reduction lever rather than a statistical nicety.

If a model improvement doubled precision from 3.0% to 6.0% while holding recall (and therefore the 240 SARs filed) fixed, the same 240 cases would require only $240/0.06 = 4{,}000$ alerts, half of the original 8,000, cutting analyst hours to $4{,}000 \times 20/60 \approx 1{,}333$ and monthly labor cost to $1{,}333 \times \$45 \approx \$60{,}000$, half of the original $120,000, since cost scales linearly with alert count once recall is held constant.

This is why a bank sets its compliance technology budget and its model-development priorities jointly: a data-science team's precision improvement translates dollar for dollar into the compliance department's staffing budget, as few other model metrics do. It is also why the layered approach of this chapter, transaction scoring (Section 15.2.3), behavioral risk tiering (Section 15.4.1), and survival-informed prioritization (Section 15.4.2), matters as much for cost as for detection accuracy. A system that better ranks alerts by true risk, even at unchanged recall, lets a fixed staff spend its hours on the alerts most likely to be real: Chapter 6's precision-recall trade-off expressed in analyst-hours and dollars.

15.6.2 Governance: Model Risk Management and the Three Lines of Defense

Chapters 8 and 9 introduced model risk management (independent validation, ongoing performance monitoring, and clear accountability for a model's use) for market and credit risk models; Chapter 14 revisited it for algorithmic trading, building on Chapter 13's Knight Capital vignette. Fraud and AML models carry an additional obligation: a poorly governed AML program exposes the institution to direct regulatory and criminal liability, not merely financial loss, since failing to detect and report money laundering is in most jurisdictions a legal violation whether or not the institution profited.

Accountability follows the three lines of defense framework of Chapter 8, unchanged: business units own the first line, an independent compliance and risk function the second, internal audit the third. What differs here is what each line is accountable *for*. The second line builds and validates this chapter's scoring and monitoring models rather than VaR models, and the third line's testing is itself subject to regulatory examination, so a weak audit function is not merely an internal control gap but a finding a supervisor can act on.

This structure answers the failure mode of the case vignette below: a monitoring system can exist on paper, and even generate large alert volumes, while failing in practice if the first line lacks the incentive or staff to act, the second line's challenge is weak, or the third line's audits are not independent of the business they review.

Model Monitoring: Precision Drift and Adversarial Adaptation

Chapter 14 introduced concept drift, a model's fitted relationship growing stale as conditions change, as a reason high-frequency trading models need frequent retraining. Fraud and AML models face a distinct version: because financial crime is adversarial (Section 15.2.1), fraudsters who learn, directly or by trial and error, which patterns a bank's model flags will shift away from them, degrading the model's precision even if the population of legitimate customers has not changed. Table 15.9 illustrates a stylized six-month trajectory for the fraud-scoring model of Section 15.2.3, holding its classification threshold fixed.

If the second line has set a precision floor of 20%, below which the alert burden of Section 15.6.1 is judged too costly relative to the fraud caught, Table 15.9 crosses it between month 3 and month 4. That should trigger a pre-defined response (refitting on recent data, adding features that capture the behavioral shift, or tightening the threshold) rather than letting the model run unchanged because no single month's decline looks alarming. This is the ongoing monitoring Chapter 8's model risk management framework requires, and the second line should define the floor in advance and act once it is breached, not leave the decision to first-line staff focused on today's alert queue.

Wulin.Suo@Queensu.ca

Month	1	2	3	4	5	6
Precision	24.2%	22.5%	20.8%	19.6%	18.3%	17.1%

Table 15.9: An illustrative six-month precision drift at a fixed classification threshold. The trajectory is constructed to show the shape, not measured from a real portfolio.

15.7 Case Vignette: The Danske Bank Estonia Case

Between roughly 2007 and 2015, the Estonian branch of Danske Bank, Denmark's largest bank, processed an estimated 200 billion euros in payments for non-resident customers, many connected to Russia and other former Soviet states, including large volumes of transactions with no plausible legitimate economic purpose. The branch had inherited this portfolio through an earlier acquisition and, rather than winding it down as a high-risk legacy business, kept serving and even expanding it, drawn by the fee income of a small branch's outsized transaction volume. A whistleblower first raised concerns in 2013; an external investigation the bank later commissioned found extensive, longstanding failures in customer due diligence, ongoing monitoring, and internal escalation of red flags, the first- and second-line failures the three-lines-of-defense framework of Section 15.6.2 is designed to prevent. Estonian authorities ultimately required the branch's closure in 2019, and in 2022 Danske Bank pleaded guilty in the United States to fraud charges connected to the case, agreeing to forfeit and pay roughly $2 billion.

The case is instructive because it was not primarily a failure of statistical modeling. Rule-based transaction monitoring of the kind Section 15.5.1 describes was reportedly in place. But it generated so many alerts relative to the staff available to review them (the problem of Section 15.6.1) that a large fraction went uninvestigated or were cleared without adequate scrutiny. It is also a case study in correspondent banking risk: much of the flagged activity moved through other banks acting as intermediaries for often hard-to-identify beneficial owners, which makes a single bank's customer due diligence, however well executed, an incomplete safeguard. The case's lasting legacy has been closer supervisory focus, across the European Union and beyond, on the governance question of Section 15.6.2: not whether a monitoring system technically exists, but whether an institution can demonstrate, to an independent third line and ultimately a regulator, that alerts are investigated and reported at a standard consistent with the risk its customer base presents.

A Current Parallel: TD Bank (2024)

The same failure recurred, at larger scale, at TD Bank, the U.S. arm of Canada's Toronto-Dominion Bank. On 10 October 2024 TD Bank, N.A. became the largest bank in U.S. history to plead guilty to Bank Secrecy Act program failures, and the first to plead guilty to conspiracy to commit money laundering. The Justice Department resolution carried about $1.89 billion in criminal fines and forfeiture; coordinated actions by FinCEN (a record $1.3 billion civil penalty), the Office of the Comptroller of the Currency, and the Federal Reserve brought the total to roughly $3 billion, with independent monitors and an OCC cap holding the bank's total U.S. assets at their 30 September 2024 level (DOJ 2024; FinCEN 2024b; OCC 2024). The failures were transaction-monitoring failures in the literal sense: from January 2018 to April 2024 the bank left 92% of its transaction volume, about $18.3 trillion, unmonitored; its monitoring scenarios went essentially unchanged from 2014 through late 2022; and three laundering networks moved more than $670 million through its accounts. Prosecutors traced this to a "flat cost paradigm" that held compliance spending flat while the bank's profits and risk grew. That is Section 15.6.1's alert economics resolved in the worst way: rather than buying precision to make the queue affordable, the bank kept it affordable by not generating alerts, and the second and third lines of Section 15.6.2 did not stop it.

15.8 Challenges in Fraud Detection, Compliance, and Financial Crime

Several challenges recur across nearly every application in this chapter.

Wulin.Suo@Queensu.ca

Extreme class imbalance makes accuracy a misleading metric. As Section 15.2.2 showed, even an accurate-sounding model produces many false positives when the event is rare, which is why this chapter, like Chapter 6, insists on precision, recall, and alert economics rather than raw accuracy.

The adversary adapts, which most of this book's other models do not have to contend with. Borrowers (Chapter 9) are not trying to defeat a credit-scoring model; a launderer engaged in structuring (Section 15.5.1) is. A fraud or AML model can therefore decay not just through the concept drift of Chapter 14 but through targeted evasion once a threshold becomes known or guessable.

Data across institutions and jurisdictions is fragmented. As the case vignette's correspondent-banking structure shows, any single institution observes only its own slice of a transaction chain that may span several banks and countries. That is why cross-institutional information sharing, within the limits of privacy and banking secrecy law, materially improves detection, and why the federated learning of Section 15.5.5 is worth its complexity.

False positives carry a real customer and reputational cost, not just an operational one. Every wrongly frozen account or declined transaction is a legitimate customer temporarily denied their own funds, a cost absent from any precision or recall statistic but material to customer relationships and, in aggregate, to regulators judging whether a program's false-positive rate is proportionate.

Explainability requirements constrain model choice. As Chapter 18 develops, a suspicious activity report or account closure frequently must be explained to a regulator or, in some jurisdictions, the customer, which limits how much a program can rely on the least interpretable models of Chapter 6, despite their potential accuracy advantage.

Governance failures, not modeling failures, are the more common root cause of large-scale enforcement actions. The Danske Bank Estonia case, like the Knight Capital vignette in Chapter 13, shows that the binding constraint is frequently not the detection model's sophistication but whether the organization around it (staffing, escalation, independent challenge, and audit) functions as the three-lines-of-defense framework of Section 15.6.2 assumes on paper.

15.9 Key Takeaways

Fraud detection and anti-money-laundering compliance apply the classification tools of Chapters 6 and 9 to a domain defined by extreme class imbalance and an adversary that adapts to known detection rules. Beyond a single logistic regression, production systems draw on the toolkit of Sections 15.2.5 and 15.2.6: class weighting and resampling such as SMOTE address the imbalance, tree-based ensembles capture nonlinear interactions an additive model misses, unsupervised anomaly detection catches patterns no confirmed label yet describes, and precision-recall AUC, not ROC-AUC, is the metric suited to a rare-event problem.

At a low base rate, even an apparently accurate model generates a large false-positive queue, and precision is far more sensitive to the false-positive rate than to the true-positive rate, so alert economics, not just accuracy, must guide how a program is designed and staffed.

A single-transaction model cannot catch a fraud that looks unremarkable in isolation, which motivates account-level risk tiering and survival analysis (the Kaplan-Meier estimator), which uses both confirmed events and censored observations to describe how account risk resolves over time.

Two growing fraud types defeat transaction-level scoring outright. A synthetic identity has no victim, so its losses are booked as credit loss and never become fraud labels, the strongest case for unsupervised detection and cross-account linkage. An authorized push payment scam is authorized by the customer, so detection must move to the customer's own baseline and the beneficiary account; mandatory reimbursement made it an expected-cost calculation, and on an instant-payment rail, when receiving-side evidence arrives late, falling back to the sender-side score is cheap while holding by default is ruinous.

Sanctions screening is a matching problem, not a lookup: a non-match can tie exactly with true matches on name similarity, so secondary identifiers must finish the job, and because liability is strict, recall cannot be traded for precision. Name screening is therefore typically a bank's largest and least precise alert queue.

Anti-money-laundering regulation adds a reporting and due-diligence apparatus (CTRs, SARs, KYC, sanctions screening) built around legal thresholds that launderers evade through placement (structuring), layering (shell-entity chains), and integration. The Danske Bank Estonia case shows that the binding

Wulin.Suo@Queensu.ca

constraint on a compliance program is at least as often organizational as statistical: governance discipline separates a monitoring system that works on paper from one that works in practice.

15.10 Suggested Exercises

15.1 A different fraud model achieves a 90% true-positive rate and a 1% false-positive rate at the same 1% fraud prevalence Chapter 1 assumed. Using Bayes' theorem, compute the resulting precision, and explain why a lower false-positive rate improved precision more than the higher true-positive rate in Chapter 1's original model.

15.2 Using the insider trading surveillance example in Section 15.2.2, recompute Pr(Insider | Flag) if the false-positive rate is reduced from 8% to 4%, holding the 2% base rate and 75% true-positive rate fixed, and compare the size of this improvement to what a similarly sized improvement in the true-positive rate alone would produce.

15.3 Using Table 15.2, suppose an eleventh transaction has an amount z-score of 2.2, a distance of 4.0 (hundreds of km), and occurs during the day (late-night flag 0). Using the fitted coefficients in Section 15.2.3, compute the transaction's predicted fraud probability and classification at a 0.5 threshold.

15.4 Using transaction 9 in Table 15.2, explain why a transaction-level model can have perfect precision and still leave a bank exposed to fraud, and how Section 15.4.1's behavioral risk tiering could catch the missed case.

15.5 Using Section 15.2.5's class-weighting example, explain why raising transaction 9's predicted probability from 33% to 41.7% did not by itself fix the false negative at the 0.5 threshold, and why class weighting plus a lower threshold reached the same operating point as lowering the unweighted model's threshold to 0.30.

15.6 Explain in a short paragraph why ROC-AUC can overstate a fraud model's practical usefulness under extreme class imbalance, and why precision-recall AUC does not share this weakness.

15.7 Using Table 15.6's method, compute the Kaplan-Meier survival estimate for a new sample of 8 flagged accounts with observed times (days, + denotes censored) of 4, 6^+, 10, 14, 14^+, 22, 28^+, 33, showing your calculation at each confirmed-fraud event time.

15.8 A customer makes four cash deposits of $9,200 each within five days. Compute the total deposited, and explain, using Section 15.5.1, why no individual deposit triggers a Currency Transaction Report although the total would if deposited at once.

15.9 Using Table 15.7's method, compute a five-hop layering chain starting from $300,000, with each entity retaining 3% and forwarding the rest, and report the amount reaching the final destination and the total retained across all five hops.

15.10 A bank's monitoring system generates 12,000 alerts per month, of which 180 become filed SARs, and each alert takes an average of 25 minutes to review. Compute the alert-to-SAR conversion rate and the total analyst-hours required per month, and compare both figures to Section 15.6.1's worked example.

15.11 Using Section 15.6.2's three-lines-of-defense framework, identify which line (first, second, or third) each of the following failures most directly represents, and why: (a) a branch employee ignores an internal escalation policy for a suspicious customer; (b) an independent audit team reports directly to the same regional manager whose unit it is auditing; (c) the compliance function never independently tests whether the transaction-monitoring model's alert thresholds are still appropriate.

Wulin.Suo@Queensu.ca

15.12 Pick two of the challenges listed in this chapter and, for each, describe a concrete scenario, not used in the text, in which it could cause a fraud or AML program to fail despite a well-designed statistical model, and explain briefly why more computing power or a more flexible model would not by itself resolve either scenario.

15.13 Explain, using the Danske Bank Estonia case vignette, why correspondent banking relationships make financial crime detection harder for any single institution, and describe one mechanism, such as cross-institution information sharing or a shared utility for sanctions and beneficial-ownership screening, that could mitigate this limitation.

15.14 Compare the class-imbalance problem in this chapter's fraud examples to the loan-default example in Chapter 6: using its confusion matrix (a 75% true-positive rate and, among the six actual non-defaulters, one false positive), compute what the loan portfolio's default rate would need to be, at the same true-positive and false-positive rates, for precision to fall to this chapter's 24.2%.

15.15 Using Greenwood's formula, compute $\widehat{\mathrm{Var}}[\hat{S}(12)]$ and the resulting 95% confidence interval for $\hat{S}(12) \approx 0.7407$, compare its width to the interval computed at $t = 20$ in the text, and explain why the earlier interval is narrower even though both come from the same 12 accounts.

15.16 Using Section 15.5.4's similarity definition, compute sim for the pairs `MOHAMMED ALRASHID`/`MUHAMMED ALRASHID` and `MOHAMMED ALRASHID`/`MOHAMMED ALRASHEED`, and explain why the second pair, which involves a different family name, nonetheless scores higher than `Muhammad Al-Rasheed` does in Table 15.8.

15.17 Using Section 15.5.4's screening-volume example, recompute the monthly alert count, precision, analyst hours, and dollar cost if a better matching engine cut the false-positive rate from 0.5% to 0.2% while holding recall at 99%, and express the annual saving relative to the $541,000 in the text. Then compute what the same 60% reduction in false positives would save in Section 15.6.1's transaction-monitoring queue, and explain why a bank might still prioritize the screening engine even though the transaction-monitoring saving is the larger.

15.18 Explain why a bank cannot manage its sanctions-screening alert volume as Section 15.2.3 manages fraud-scoring alerts, by raising the threshold, and describe what a screening program can change instead to reduce false positives without reducing recall.

15.19 Using Section 15.3's cultivation arithmetic, recompute the bust-out for a synthetic identity that grows *six* tradelines to a $7,500 limit each over 24 months while paying $150 a month across all of them. Report the amount paid in, the exposure at bust-out, the net loss, and the return on the fraudster's outlay, and explain why the return multiple is lower than the text's despite the larger loss.

15.20 Suppose the reimbursement rules of Section 15.3 were tightened so the sending bank bore 100% of authorized push payment losses rather than half. Holding the 500,000 monthly payments, 0.02% scam rate, $4,200 average loss, 60% recall, 5% precision, 15-minute intervention and $45 hourly cost fixed, recompute the loss prevented, the net monthly benefit, and the break-even recall, and comment on what the change implies about how much detection effort a bank would rationally fund under each regime.

15.21 Using Figure 15.1 and the operating points quoted with it, compute the precision of each model at a false-positive rate of 1%, where logistic regression reaches a true-positive rate of 0.294 and gradient boosting 0.412 on the 34 frauds and 966 legitimate transactions. Compare the result with the two models' ROC-AUC ranking, and explain which ranking a fraud team that can review about 25 alerts per 1,000 transactions should trust.

15.22 Using Section 15.3.2's real-time scoring example, recompute the sender-side log-odds and scam probability for the same payment if a phone call *is* in progress during the session, and decide whether the payment should be held before the payee lookup returns. Then recompute the hold threshold p^* if a call stops only 50% of scams, and state whether the decision changes. Finally, using Table 15.4,

Wulin.Suo@Queensu.ca

explain why holding every payment whose payee lookup times out is worse than releasing every payment.

15.23 **Challenge (real data).** Download the Kaggle "Credit Card Fraud Detection" dataset (Université Libre de Bruxelles; roughly 285,000 European cardholder transactions over two days, with 492 confirmed frauds and features already anonymized via PCA). Its 0.172% fraud rate is far more extreme than Table 15.2's toy example. (a) Fit a logistic regression with and without class weighting, as in Section 15.2.5, and report precision, recall, F_1, and precision-recall AUC for both, since accuracy is uninformative at this imbalance. (b) Using Section 15.6.1's cost-based framework, with a \$50 review cost per flagged transaction and a \$200 average loss per missed fraud, find the threshold that minimizes total expected cost, and compare its alert volume to a naive 0.5 threshold's. (c) Since the features are anonymized PCA components, this dataset cannot support the domain-driven feature engineering (for example, distance from home) of this chapter's toy example; discuss what information a real bank's fraud team has that this privacy-scrubbed dataset withholds.

15.11 Further Reading

- Bolton and Hand (2002), "Statistical Fraud Detection: A Review." A foundational survey of statistical approaches to fraud detection, including the class-imbalance and base-rate issues central to Section 15.2.2.
- Chawla et al. (2002), "SMOTE: Synthetic Minority Over-sampling Technique." The original paper introducing the resampling technique discussed in Section 15.2.5.
- Klein and Moeschberger (2003), *Survival Analysis*. A standard, thorough reference on the Kaplan-Meier estimator and censoring introduced in Section 15.4.2.
- FATF (2012), *International Standards on Combating Money Laundering and the Financing of Terrorism & Proliferation*. The primary international regulatory framework underlying the KYC, CDD, and reporting obligations discussed in Section 15.5.1.
- OFAC (2019), *A Framework for OFAC Compliance Commitments*. The regulator's own statement of what a sanctions compliance program must include, and the primary source for the strict-liability standard underlying Section 15.5.4.
- FinCEN (n.d.), *Bank Secrecy Act Regulations and Guidance*. The primary U.S. regulatory source for the CTR and SAR reporting requirements discussed in this chapter.
- Bruun & Hjejle (2018), *Report on the Non-Resident Portfolio at Danske Bank's Estonian Branch*. The external investigation commissioned by Danske Bank itself, the primary source for the factual account in this chapter's case vignette.
- IIA (2020), *The IIA's Three Lines Model*. The current authoritative statement of the governance framework developed in Section 15.6.2.

Wulin.Suo@Queensu.ca